Ateneo de Davao University

4-334 · Full-Stack Web Development · 1st Semester, July–November 2026

Lab Activity 10 – Barangay Bulletin: Eloquent, Migrations & Seeding

Week 10 topics: ORM / ActiveRecord, Eloquent models ($fillable, create / find / save), migrations (up / down, rollback), seeders, Faker factories (states), query builder filters, pagination.
Submission deadline: _____ **Repository:** `LabActivity10-Lastname` (public)

Scenario

The **Barangay Matina Crossing Hall** in Davao City still posts announcements (clean-up drives, Bakuna schedules, curfew reminders, ayuda payouts) on a cork board outside the hall. Kapitan wants an online **Barangay Bulletin**, a small blog where barangay officials (authors) publish posts that residents can browse, search and page through. Last week you built the Blade + Tailwind presentation layer; this week every page must read **real rows from the database through Eloquent**. Relationships (`hasMany` / `belongsTo`) are Week 11, so this lab does **not** require them.

Personalize it:

- The site title must be "*Barangay Bulletin – Purok [your last name]*".
- One author must be "*Kagawad [your first name] [your last name]*", email `kagawad.[lastname]@bulletin.test`, created by hand with `Author::create()` (not by the factory).
- One pinned post titled "*Purok [Lastname] Clean-up Drive*", written by your Kagawad, with `published_at` on your birthday in 2026.
- **Posts per page** = the number of letters in your last name (use 6 if your last name is shorter than 6 letters). Example: *Estremos* → 8 per page.
- Post categories must be exactly: *Announcement, Health, Peace & Order, Ayuda*, plus one category of your choice.

Setup: New Laravel 12 Project with SQLite

Step 1 – Create the project (Laravel 12 uses SQLite by default)

```
composer create-project laravel/laravel LabActivity10-Lastname
cd LabActivity10-Lastname
npm install
```

Step 2 – Check the database settings in `.env`

```
DB_CONNECTION=sqlite
# DB_DATABASE line stays commented out; Laravel uses database/database.sqlite
```

If `database/database.sqlite` does not exist, create it: `touch database/database.sqlite` (Windows PowerShell: `New-Item database/database.sqlite`).

Step 3 – Tailwind. Reuse your Week 9 setup. Either the Tailwind v3 + PostCSS setup from Lab 9 or Laravel 12's default Tailwind v4 (`@tailwindcss/vite`) is accepted this week. Keep `@vite([...])` in your layout and copy your `layouts/app.blade.php` from Lab 9 as a starting point.

Step 4 – Generate the files with Artisan (`-mf` creates the model, migration and factory together)

```
php artisan make:model Author -mf
php artisan make:model Post -mf
php artisan make:seeder AuthorSeeder
php artisan make:seeder PostSeeder
php artisan make:controller PostController
php artisan make:migration add_views_to_posts_table --table=posts
```

Part A – Migrations

Step 5 – `create_authors_table`. Authors must be created **before** posts, so the authors migration's timestamp must be earlier (it is, if you ran the commands in order).

```
public function up(): void
{
    Schema::create('authors', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('position');          // e.g. Kagawad, Barangay Secretary, BHW
        $table->string('email')->unique();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('authors');
}
```

Step 6 – `create_posts_table`

```
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('author_id')->constrained()->cascadeOnDelete();
    $table->string('title');
    $table->string('category');
    $table->text('body');
    $table->boolean('is_published')->default(false);
    $table->boolean('is_pinned')->default(false);
```

```
    $table->timestamp('published_at')->nullable();
    $table->timestamps();
});
// down(): Schema::dropIfExists('posts');
```

Step 7 – A second, separate migration. Kapitan now wants a view counter. Do **not** edit the posts migration you already ran; add the column in `add_views_to_posts_table`:

```
public function up(): void
{
    Schema::table('posts', function (Blueprint $table) {
        $table->unsignedInteger('views')->default(0)->after('body');
    });
}

public function down(): void
{
    Schema::table('posts', function (Blueprint $table) {
        $table->dropColumn('views');
    });
}
```

Run `php artisan migrate`, then prove your `down()` methods work with `php artisan migrate:rollback` followed by `php artisan migrate` again.

Part B – Models (ActiveRecord)

Step 8 – `app/Models/Post.php`. Without `$fillable`, `create([...])` throws a MassAssignmentException.

```
class Post extends Model
{
    use HasFactory;

    protected $fillable = ['author_id', 'title', 'category', 'body',
                           'is_published', 'is_pinned', 'published_at', 'views'];

    protected $casts = [
        'is_published' => 'boolean',
        'is_pinned'    => 'boolean',
        'published_at' => 'datetime',
    ];
}
```

Do the same for `Author` (`name`, `position`, `email`).

Part C – Factories & Seeders

Step 9 – `database/factories/PostFactory.php` with a `draft()` state:

```
public function definition(): array
{
    return [
        'author_id'    => Author::factory(),
        'title'        => fake()->sentence(6),
        'category'     => fake()->randomElement(['Announcement', 'Health',
                              'Peace & Order', 'Ayuda', 'YOUR-CATEGORY']),
        'body'         => fake()->paragraphs(3, true),
        'is_published' => true,
        'published_at' => fake()->dateTimeBetween('-3 months', 'now'),
    ];
}

public function draft(): static
{
    return $this->state(fn () => ['is_published' => false, 'published_at' => null]);
}
```

Write `AuthorFactory` yourself (name, position from a list such as Kagawad / Secretary / BHW / Tanod, unique safe email). Tip: `fake('en_PH')` is not guaranteed to exist; plain `fake()` is fine.

Step 10 – `AuthorSeeder`: create your Kagawad by hand with `Author::create([...])`, then `Author::factory()->count(4)->create()`.

Step 11 – `PostSeeder`: **50** factory posts spread over the existing authors (40 published, 10 drafts), plus your pinned post.

```
$authors = Author::all();

Post::factory()->count(40)
    ->sequence(fn ($seq) => ['author_id' => $authors->random()->id])
    ->create();

Post::factory()->count(10)->draft()
    ->sequence(fn ($seq) => ['author_id' => $authors->random()->id])
    ->create();

// Your pinned post: write this one with Post::create([...])
// (author = your Kagawad, is_pinned = true, published_at = your 2026 birthday)
```

Step 12 – `DatabaseSeeder`

```
public function run(): void
{
    $this->call([
        AuthorSeeder::class,
        PostSeeder::class,
    ]);
}
```

Rebuild and verify:

```
php artisan migrate:fresh --seed
php artisan tinker
>>> App\Models\Author::count()                    // 5
>>> App\Models\Post::count()                      // 51
>>> App\Models\Post::where('is_published', false)->count()   // 10
```

Step 13 – ActiveRecord in tinker. In the same tinker session, run and **screenshot** this (save as `docs/tinker.png`):

```
>>> $post = App\Models\Post::find(1);
>>> $post->title = 'Purok [Lastname]: ' . $post->title;
>>> $post->save();
>>> App\Models\Post::find(1)->title
```

Part D – Query Builder, Pagination & Views

Step 14 – Routes (`routes/web.php`)

```
use App\Http\Controllers\PostController;

Route::redirect('/', '/posts');
Route::get('/posts', [PostController::class, 'index'])->name('posts.index');
Route::get('/posts/{id}', [PostController::class, 'show'])->name('posts.show');
```

Step 15 – `PostController@index`. Starter shape; you must finish the filters:

```
public function index(Request $request)
{
    $posts = Post::query()
        ->where('is_published', true)
        // TODO: if $request->filled('q'): where title LIKE "%q%"
        // TODO: if $request->filled('category'): where category = ...
        ->orderByDesc('is_pinned')
        ->latest('published_at')
        ->paginate(PER_PAGE)          // your last-name letter count
        ->withQueryString();          // keeps ?q= and ?category= on page links

    $categories = Post::query()->distinct()->orderBy('category')->pluck('category');

    return view('posts.index', compact('posts', 'categories'));
}
```

Step 16 – `PostController@show`: load one post with `Post::findOrFail($id)` (a draft must return 404), add 1 to `views` and call `save()`, then return `view('posts.show', compact('post'))`.

Step 17 – `resources/views/posts/index.blade.php` (extends your layout):

```
<form method="GET" action="{{ route('posts.index') }}" class="flex flex-col sm:flex-row gap-2">
    <input name="q" value="{{ request('q') }}" placeholder="Search title">
    <select name="category"> ... @foreach ($categories as $c) ... @endforeach </select>
    <button>Filter</button>
```

```
</form>

@forelse ($posts as $post)
    ... title (link to posts.show), category badge, published date, views,
        a "Pinned" badge when $post->is_pinned ...
@empty
    <p>No posts match your search.</p>
@endforelse

{{ $posts->links() }}
<p>Showing {{ $posts->firstItem() }}–{{ $posts->lastItem() }} of {{ $posts->total() }} posts</p>
```

Laravel's default pagination view is Tailwind-styled, so `links()` needs no extra package. The author's name is **not** required on cards yet (that needs a relationship, Week 11); showing the author ID is fine.

Troubleshooting

- "**no such table: posts**": run `php artisan migrate:fresh --seed`; check `DB_CONNECTION=sqlite` and that `database/database.sqlite` exists.
- **FOREIGN KEY constraint failed**: the posts migration ran before authors, or a post got an `author_id` that doesn't exist. Check the migration file timestamps and that `AuthorSeeder` is called first.
- **MassAssignmentException / columns saved as NULL**: the attribute is missing from `$fillable`.
- **Page 2 loses the search**: add `->withQueryString()` after `paginate()`.
- "**Column not found: views**": you edited the old migration instead of running the new one. Run `php artisan migrate`.

Requirements

1. **Migrations with up/down.** `authors` and `posts` migrations using Blueprint helpers (`id`, `string`, `text`, `foreignId()->constrained()`, `timestamps`), each with a working `down()`. `migrate:rollback` then `migrate` runs cleanly. No raw `CREATE TABLE` SQL.
2. **Separate alter migration.** `views` is added by its own `add_views_to_posts_table` migration using `Schema::table`, whose `down()` drops the column. The original posts migration does not contain `views`.
3. **Eloquent models.** `Author` and `Post` models with explicit `$fillable` (no `$guarded = []`) and casts for the booleans and `published_at`. `docs/tinker.png` shows `find()` → change attribute → `save()`.
4. **Factories with Faker + a state.** `AuthorFactory` and `PostFactory` use `fake()`; `PostFactory` has a `draft()` state and uses only your five categories.
5. **Seeders.** `DatabaseSeeder` calls `AuthorSeeder` then `PostSeeder` via `$this->call()`. After `php artisan migrate:fresh --seed`: 5 authors (your Kagawad made with `Author::create`), 51 posts (40 published, 10 drafts, plus your pinned birthday post made with `Post::create`).
6. **Query builder filters.** `/posts` shows only published posts, pinned first then newest, and supports `?q=` (title LIKE search) and `?category=`, each applied only when filled, through query builder methods (no string-concatenated SQL, no `DB::select`).

7. **Pagination.** `paginate(N)` with N = letters in your last name (min 6), `withQueryString()`, `{{ $posts->links() }}` and a "Showing x–y of z" line. Never `get()` all posts for the list.
8. **Show page with save().** `/posts/{id}` uses `findOrFail`, returns 404 for missing or draft posts, and increments `views` through `save()` (or `increment()`); the count shows on the page and in the list.
9. **Blade + Tailwind views.** Both pages extend your layout (title "Barangay Bulletin – Purok Lastname"), use `@forelse … @empty` ("No posts match your search."), a Pinned badge, and a responsive filter form/card grid with at least one `sm:`/`md:` prefix. All output escaped with `{{ }}`.
10. **Authenticity.** Code, commits and README must look like your own work: meaningful commit history (not a single dump), your personalized data, and a README listing: your per-page number and why, your custom category, your birthday post date, and how to run it (`composer install`, `cp .env.example .env`, `php artisan key:generate`, `php artisan migrate:fresh --seed`, `npm install`, `npm run dev`, `php artisan serve`).

Deliverables

- Public GitHub repo `LabActivity10-Lastname` with the full Laravel project (no `vendor/`, `node_modules/` or `database.sqlite`), including `docs/tinker.png`.
- Submit the repo link on Moodle before the deadline. Be ready to defend your work in lab.

Rubric (10 points)

#	Criterion	Pts
1	authors/posts migrations with Blueprint helpers, FK constrained, working down(); rollback OK	1
2	Separate add_views migration via Schema::table with reversible down()	1
3	Models with $fillable + casts; tinker screenshot of find → save	1
4	Faker factories with draft() state and the required categories	1
5	Seeders via $this->call(); migrate:fresh --seed gives 5 authors / 51 posts incl. personal rows	1
6	Published-only, pinned-first list with conditional q/category query builder filters	1
7	paginate(N personal) + withQueryString + links() + showing x–y of z	1
8	Show page: findOrFail, 404 on drafts, views incremented with save()	1
9	Blade/Tailwind index + show: layout, @forelse/@empty, Pinned badge, responsive, escaped	1
10	AI authenticity: 1 = looks student-written, 0.5 = mixed, 0 = strongly AI-looking	1
	Total	**10**

Notes (not scored): lab presence and a public repository are required for the work to be checked. Scores are rounded down to a whole number.